

7. Корисні фішки Git

Команда	Для чого
<code>git log --graph --oneline</code>	візуальна історія гілок
<code>git show <hash></code>	деталі конкретного коміту
<code>git blame file.py</code>	показує, хто змінював рядки
<code>git checkout <hash></code>	перейти до старої версії
<code>git revert <hash></code>	створює коміт, який «відмінює» інший

Пояснення до кожної

`git log --graph --oneline`

```
git log --graph --oneline --all --decorate
```

Малює дерево гілок прямо в терміналі:

```
* f1a2b3c (HEAD -> main) merge feature-login
| \
| * 9d8c7b6 (feature-login) add login form
* | 4e5f6a7 fix typo in readme
| /
* 1234567 initial commit
```

`git show <hash>`

```
git show 3d4e133      # повний хеш вводити не треба — перших 7 символів досить
git show 3d4e133 --stat # лише перелік змінених файлів
git show HEAD         # останній коміт
git show HEAD~1       # попередній коміт
```

`HEAD` — це «де я зараз». `HEAD~1` — на один коміт назад, `HEAD~3` — на три.

`git blame file.py`

```
git blame main.py
# 3d4e133 (Irop 2026-08-18 12:37:00 +0300 1) print("Hello Git!")
```

Показує для **кожного рядка**: хто його написав, коли і в якому коміті. Потрібно не щоб когось звинувачувати, а щоб зрозуміти, **чому** рядок такий: знайшов коміт → прочитав його повідомлення → зрозумів задачу.

`git checkout <hash>` — подивитися минуле

```
git checkout 1234567    # перейти до старої версії
git switch -            # повернутись туди, де був
git checkout main       # або явно назвати гілку
```

Після цього Git скаже `You are in 'detached HEAD' state` — це означає «ти дивишся історію, а не працюєш у гілці». Просто дивись, не комітити. Якщо хочеш продовжити роботу з цієї точки — створи гілку:

```
git checkout -b experiment-from-old-version
```

`git revert <hash>` — безпечне скасування

```
git revert 3d4e133      # створює НОВИЙ коміт, який відмінняє зміни того
```

Головна відмінність від `reset` :

	<code>git revert</code>	<code>git reset --hard</code>
Що робить	додає коміт-протилежність	стирає коміти з історії
Історія	зберігається повністю	переписується
Чи можна після push	так, безпечно	ні (зламає роботу колегам)

Тому в командній роботі скасовують саме через `revert` .

Ще кілька речей, які роблять життя легшим

Псевдоніми (alias) для довгих команд

```
git config --global alias.st status
git config --global alias.lg "log --graph --oneline --all --decorate"
# тепер працює: git st   i   git lg
```

Пошук по історії

```
git log --grep="login"      # коміти, у повідомленні яких є «login»
git log -S "def add"        # коміти, де з'явився або зник цей код
git log --author="Igor"     # коміти конкретного автора
git log -- main.py          # історія одного файлу
```

Порятунок після невдалого експерименту

```
git reflog                  # ВСЬ журнал переміщень HEAD, навіть «втрачені» коміти
git checkout <hash з reflog> # повернутись до нього
```

`git reflog` — головна команда з розряду «я все зламав». Поки коміт існував хоча б раз, Git тримає його ще ~30 днів, тому втратити роботу дуже важко.